

DD: BIL-03 — Cycle Close

Developer implementation guide: DD_BIL-03-Cycle_Close-DEV_IMPL_GUIDE-v1.1.md.

1. Purpose

This rewritten DD is the developer-facing version of the module contract DD_BIL-03-Cycle_Close-v1.1.md.

Verdict: ■ New | **Wave:** 1 | **Group II:** Billing & Revenue **Orchestration:** Scheduled scanner jobs + synchronous service classes per SUB-WF-FRAMEWORK-01 (Java/Spring OR Laravel/PHP per-module choice). No Camunda — BIL-03 is a recurring batch job, not a workflow.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/05_billing/DD_BIL-03-Cycle_Close-v1.1.md
Version	v1.1
DD type	module contract
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- GET /api/billing/charges/{cycle_charge_invoice_id}/status
- GET /api/cycle-close-runs/{run_id}
- GET /api/cycle-close-runs?limit=50
- GET /api/cycle-evaluations/failed
- GET /api/cycle-evaluations/{subscriptionId}
- GET /api/cycle-evaluations?evaluation_type={type}&next_before={ts}&limit=50
- GET /api/wallets/{subscription_id}/balance
- POST /api/cycle-evaluations/{subscriptionId}/force-close
- POST /api/cycle-evaluations/{subscriptionId}/reset-failures

- POST /api/cycle-evaluations/{subscriptionId}/skip-current
- POST /api/cycles/{subscriptionId}/advance-cycle-on-recovery
- POST /api/cycles/{subscriptionId}/realign-on-late-payment
- POST /api/cycles/{subscription_id}/advance-cycle-on-recovery
- POST /api/cycles/{subscription_id}/realign-on-late-payment
- POST /api/wallets/{subscription_id}/debit

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- advance-cycle-on-recovery
- billing-cycle-close
- put-master-fields
- realign-on-late-payment

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- InvoiceOverdue
- PaymentApplied
- SubscriptionActivated
- SubscriptionDowngraded
- SubscriptionMigrated
- SubscriptionPaused
- SubscriptionRelocated
- SubscriptionResumed
- SubscriptionTerminated
- SubscriptionUpgraded

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 1 | **Group II:** Billing & Revenue

Orchestration: Scheduled scanner jobs + synchronous service classes per SUB-WF-FRAMEWORK-01 (Java/Spring OR Laravel/PHP per-module choice). No Camunda — BIL-03 is a recurring batch job, not a workflow.

A — Target

1. What this process is about

BIL-03 is the **cycle close engine** — the module that detects when a Subscription's billing cycle has reached its end (or, for pre-payment, its start), and triggers the appropriate downstream actions. It runs continuously: scheduled scanners periodically evaluate every active Subscription against its cycle anchor, and at each cycle boundary fire the events and module coordination required by the Subscription's billing model.

For **postpaid** Subscriptions (Wananchi KE/UG/TZ): at cycle end, BIL-03 emits `CycleClosed` so BIL-02 can generate the cycle's invoice. The customer's billing for what they consumed during the cycle is finalized at this point.

For **pre-payment** Subscriptions (Yas SN): at cycle boundary, BIL-03 checks whether the upcoming cycle has been paid. If yes, it emits `CycleActivated` and the new cycle begins. If no, it emits `CyclePaymentMissed` and BIL-04 enters the Subscription into dunning. The customer's billing model demands prepayment-before-activation.

BIL-03 is fundamentally a **per-Subscription cycle anchor tracker**, with the anchor and cycle window owned by SUB-LM-01 (`cycle_model`, `cycle_anchor_day`, `cycle_frequency_months`, etc. — already documented in SUB-LM-01 and referenced in the canonical billing envelope per SUB-WF-FRAMEWORK-01). BIL-03's job is to detect the boundary, fire the right events, and update the Subscription's cycle anchor for the next cycle.

Out of scope:

- **Generating invoices** — owned by BIL-02 (invoicing). BIL-03 emits `CycleClosed`; BIL-02 consumes and generates the invoice.
- **Computing what's owed** — owned by BIL-01 (charging engine). Charges accumulated during the cycle are queried by BIL-02 at invoice generation; BIL-03 itself does not compute amounts.
- **Detecting unpaid invoices post-due-date** — owned by BIL-02 (invoicing). BIL-02 evaluates due dates after invoice generation and emits `InvoiceOverdue` when grace period expires.
- **Driving the dunning state machine** — owned by BIL-04 (dunning). BIL-03 emits `CyclePaymentMissed` for pre-payment; BIL-04 consumes and creates the dunning state.
- **Cycle anchor configuration** — owned by SUB-LM-01. Cycle attributes are set at Subscription activation by SUB-WF-ACTIVATE-01 and can be adjusted by SUB-WF-UPGRADE-01 / DOWNGRADE-01 / RELOCATION-01 / MIGRATION-01 as documented in those DDs.
- **Money math, prorate, settlement** — owned by BIL-01.
- **Wallet balance, topup processing, wallet debit operations** — owned by BIL-05 (Wallet & Topup, pending DD). BIL-03 queries BIL-05 for balance and calls BIL-05 for debit at cycle boundary for PREPAID Subscriptions, but does not own wallet state.
- **Notification dispatch** (cycle-end reminders, payment-required notices, etc.) — owned by notification module. BIL-03 emits Kafka events at each cycle boundary; notification consumes.
- **Mid-cycle one-off charges** — owned by BIL-01. These don't affect cycle close; they're settled at cycle close along with the regular cycle charge.

2. Business rules

Rules grouped by scope: C for cycle close logic, E for evaluation cadence and scheduling, P for pre-payment specifics, S for state interactions, V for cycle alignment on workflow events.

Cycle close logic rules

ID	Rule	Triggered on
R-BIL-03-C-1	A Subscription's cycle boundary is the moment when its current cycle ends and the next begins. The boundary timestamp is computed from the Subscription's <code>cycle_model</code> , <code>cycle_anchor_day</code> , <code>cycle_frequency_months</code> , and <code>cycle_period_days</code> fields (owned by SUB-LM-01). BIL-03 reads these from SUB-LM-01 via REST and does not store its own cycle anchor copy.	Cycle boundary detection
R-BIL-03-C-2	The cycle boundary action depends on the Subscription's <code>billing_mode</code> : POSTPAID → fire <code>CycleClosed</code> event (for BIL-02 to generate the invoice for what was consumed during the cycle just ended); PREPAID → query BIL-05 for wallet balance, then if sufficient debit wallet for cycle charge and fire <code>CycleActivated</code> , else fire <code>CyclePaymentMissed</code> ; PREPAYMENT → check whether the upcoming-cycle invoice has been paid (queried via BIL-01), then fire either <code>CycleActivated</code> (paid) or <code>CyclePaymentMissed</code> (unpaid). All three modes are active in V3.	Cycle boundary fire

ID	Rule	Triggered on
R-BIL-03-C-3	After firing the cycle event, BIL-03 advances the Subscription's cycle anchor: current_cycle_start = old_cycle_end + 1 unit, current_cycle_end = new start + cycle_period. This is a put-master-fields call to SUB-LM-01 (BIL-03 does not write directly to the subscription table). The anchor is advanced only on success paths: POSTPAID CycleClosed, PREPAID CycleActivated (after successful wallet debit), and PREPAYMENT CycleActivated. For PREPAID and PREPAYMENT with CyclePaymentMissed, the anchor is NOT advanced — the cycle stays "stuck" at the missed boundary until payment arrives (BIL-04's auto-recovery path on WalletToppedUp for PREPAID per R-BIL-04-R-8, or BIL-01's late-payment realignment for PREPAYMENT per R-BIL-04-R-6, will call BIL-03's realign-on-late-payment endpoint to advance the anchor at that point).	Cycle anchor advance
R-BIL-03-C-4	Cycle close on a Subscription in any of these statuses is a no-op : CREATED (not yet activated), TERMINATED, RETIRED, PENDING_* (mid-workflow). For SUSPENDED (either voluntary or non-payment), cycle close is skipped — no event fired, no anchor advance — and the next evaluation is scheduled to the next cycle boundary. SUSPENDED Subscriptions resume their cycle when SUB-WF-RESUME-01 runs (which fixes the anchor as part of the resume workflow).	Cycle close gating
R-BIL-03-C-5	Each cycle close action is idempotent on (subscription_id, cycle_window_end). BIL-03 records last_cycle_closed_at and last_cycle_closed_window_end on the Subscription (via SUB-LM-01 put-master-fields). Repeated scanner runs that find the same cycle_window_end already recorded as closed are no-ops. This makes scanner-restart safe across node crashes.	Idempotency

Evaluation cadence rules

ID	Rule	Triggered on
R-BIL-03-E-1	A cycle close scanner runs every 30 minutes (configurable per environment), scanning Subscription rows where: status IN (ACTIVE) AND current_cycle_end <= now() + clock_skew_buffer AND (last_cycle_closed_window_end IS NULL OR last_cycle_closed_window_end < current_cycle_end). For each matched Subscription, BIL-03 invokes cycle close synchronously.	Scheduler
R-BIL-03-E-2	A pre-payment pre-notice scanner runs daily (configurable, default 03:00 local-time per tenant), scanning Subscriptions with billing_mode = PREPAYMENT AND current_cycle_end <= now() + pre_notice_window_days AND no pre-notice already fired for this cycle. For each match, BIL-03 emits CyclePaymentDueSoon Kafka event (consumed by notification for the "your cycle starts in N days — pay now" SMS/email). Default pre-notice window: 3 days.	Daily pre-notice
R-BIL-03-E-3	Per-Subscription serialization: a SELECT FOR UPDATE of the Subscription's cycle_lock advisory lock (or row lock on the Subscription itself for cycle-management operations) at scanner start ensures only one node processes a given Subscription at a time. The same advisory lock is held by SUB-WF-* workflows when they mutate cycle attributes, preventing race conditions.	Scanner

ID	Rule	Triggered on
R-BIL-03-E-4	Batch size: each scanner run processes at most 1,000 Subscriptions per cycle to prevent monopolizing downstream modules (BIL-02 invoice generation, BIL-04 dunning, notification dispatch). Subscriptions not processed in a given run are evaluated in subsequent runs based on <code>current_cycle_end</code> ordering.	Scanner
R-BIL-03-E-5	Cycle close runs are tracked in <code>cycle_close_run</code> table with <code>run_id</code> , <code>started_at</code> , <code>completed_at</code> , <code>subscriptions_evaluated</code> , <code>subscriptions_closed</code> , <code>subscriptions_skipped</code> , <code>subscriptions_failed</code> . Failed Subscriptions are logged with <code>last_cycle_close_failure_code</code> and retry on the next scanner run. After 3 retry failures, the Subscription is flagged for operator review via <code>CycleCloseFailed</code> Kafka event.	Audit + observability

Pre-payment specific rules

ID	Rule	Triggered on
R-BIL-03-P-1	A pre-payment cycle is paid when there is a successful payment recorded against the Subscription's <code>next_cycle_charge_invoice_id</code> (the invoice/charge BIL-02 issued for the upcoming cycle). BIL-03 queries BIL-01 via REST at cycle boundary to determine payment status: <code>GET /api/billing/charges/{cycle_charge_invoice_id}/status</code> returns <code>PAID / UNPAID / PARTIALLY_PAID</code> . Only <code>PAID</code> activates the cycle; <code>UNPAID</code> and <code>PARTIALLY_PAID</code> both result in <code>CyclePaymentMissed</code> .	Pre-payment evaluation
R-BIL-03-P-2	For a pre-payment subscription, BIL-02 generates the cycle charge invoice in advance (at the prior cycle's close minus <code>pre_invoicing_window_days</code> , default 7 days). The invoice ID is recorded on the Subscription's <code>next_cycle_charge_invoice_id</code> field (BIL-02 owns the write via SUB-LM-01 master fields). BIL-03 reads this field to know which invoice to check at cycle boundary.	Pre-payment cycle invoice setup
R-BIL-03-P-3	A pre-payment cycle that fires <code>CyclePaymentMissed</code> has its cycle anchor frozen at the missed boundary . The next evaluation is set to <code>current_cycle_end + 24 hours</code> (BIL-04's dunning scanner runs more frequently and will re-evaluate); BIL-03 does not advance the cycle itself. The cycle "unfreezes" only when BIL-01 emits <code>PaymentApplied</code> and BIL-04's recovery path realigns the anchor (per R-BIL-04-R-6).	Cycle freeze on missed payment
R-BIL-03-P-4	If a pre-payment cycle's invoice gets paid after the cycle boundary but before dunning escalates to suspension (level 3): BIL-01 calls BIL-03's internal endpoint <code>POST /api/cycles/{subscription_id}/realign-on-late-payment</code> with the payment timestamp. BIL-03 advances the cycle anchor to start at the payment timestamp, emits <code>CycleActivated</code> with <code>lateActivation: true</code> and <code>originalCycleStart</code> for audit.	Late payment in pre-payment
R-BIL-03-P-5	The pre-payment cycle is NOT retroactively serviced if paid late. Per R-BIL-04-R-6: the customer pays for a fresh cycle starting from payment date; the period from the missed cycle start to payment date is forfeited (not refunded, not back-billed, not serviced). This is the standard pre-payment recovery model.	Late payment semantics

PREPAID wallet specific rules

ID	Rule	Triggered on
R-BIL-03-W-1	A PREPAID Subscription's cycle close evaluation: BIL-03 queries BIL-05 via REST for current wallet balance (GET /api/wallets/{subscription_id}/balance) and computes the required cycle charge (from the Subscription's package + any pending mid-cycle additions). If wallet_balance >= cycle_charge, BIL-03 calls BIL-05 POST /api/wallets/{subscription_id}/debit with the cycle charge amount and an idempotency key cycle-debit-{subscription_id}-{cycle_end_iso}. On successful debit, BIL-03 fires CycleActivated. On insufficient balance, BIL-03 fires CyclePaymentMissed (same event as PREPAYMENT — uniform dunning trigger) and does NOT advance the cycle anchor.	PREPAID cycle evaluation
R-BIL-03-W-2	BIL-05 wallet debit is atomic and idempotent on the supplied idempotency key — repeat scanner calls with the same key after a previous success return the cached prior result and do not double-debit. Wallet debit failures (BIL-05 unavailable, internal error) are retried on the next scanner pass; the cycle is not closed until BIL-05 confirms the debit. After 3 retries, BIL-03 fires CycleCloseFailed for operator review.	Wallet debit reliability
R-BIL-03-W-3	No partial wallet debit. If wallet balance is less than the cycle charge, BIL-03 fires CyclePaymentMissed and waits — it does NOT debit the partial amount and request the remainder later. The wallet stays untouched until either a topup brings it above the threshold (BIL-04 picks this up via WalletToppedUp and triggers recovery, R-BIL-04-R-8) or the dunning escalation completes.	Wallet partial-balance handling
R-BIL-03-W-4	For a PREPAID Subscription with CyclePaymentMissed, the cycle anchor stays frozen. When BIL-04's PREPAID recovery (R-BIL-04-R-8) succeeds — wallet debited via BIL-05, anchor advance required — BIL-04 calls BIL-03's internal endpoint POST /api/cycles/{subscription_id}/advance-cycle-on-recovery (separate from the PREPAYMENT realign-on-late-payment endpoint) with the recovery timestamp. BIL-03 advances the cycle anchor to start at the recovery timestamp, emits CycleActivated with recoveryFromDunning: true and originalCycleStart for audit.	PREPAID cycle advance on recovery
R-BIL-03-W-5	Pre-cycle wallet pre-notice: a daily scanner (the same PrePaymentPreNoticeScanner per R-BIL-03-E-2, extended) also checks PREPAID Subscriptions. For each PREPAID Subscription with current_cycle_end <= now() + pre_notice_window_days AND wallet balance < cycle charge: emits WalletLowBeforeCycle Kafka event (consumed by notification for the "your wallet is low — top up X KES before [date] to avoid service interruption" SMS/email). The wallet balance check is a real-time query to BIL-05 at scan time.	Pre-cycle wallet warning

State interaction rules

ID	Rule	Triggered on
R-BIL-03-S-1	When SUB-WF-PAUSE-01 pauses a Subscription, the in-flight cycle is prorated to the pause start date if the billing program's pauses_bill_cycle flag is TRUE (per SUB-WF-PAUSE-01 rules). BIL-03 receives the SubscriptionPaused event and: emits CyclePartiallyClosed with the prorated window (BIL-02 generates a prorated invoice), updates the Subscription's cycle anchor to "frozen pending resume" state.	SubscriptionPaused event

ID	Rule	Triggered on
R-BIL-03-S-2	When SUB-WF-RESUME-01 resumes a Subscription (either voluntary or DUNNING_RESUME), the cycle anchor is set per SUB-WF-RESUME-01's rules. BIL-03 receives the SubscriptionResumed event and reads the updated cycle anchor from SUB-LM-01. The new cycle's evaluation is scheduled at <code>current_cycle_end</code> .	SubscriptionResumed event
R-BIL-03-S-3	When SUB-WF-UPGRADE-01 / DOWNGRADE-01 immediately changes a Subscription's package mid-cycle, the cycle anchor may be re-aligned per the workflow's rules (typically NOT changed — the current cycle continues, with the prorate delta charged or credited within the cycle). BIL-03 just reads the updated cycle window on the next scanner evaluation; no special handling needed.	Mid-cycle upgrade/downgrade
R-BIL-03-S-4	When SUB-WF-DOWNGRADE-01 schedules a NEXT_CYCLE downgrade, BIL-03 does NOT execute the downgrade — that's owned by the scheduler in SUB-WF-FRAMEWORK-01 (<code>scheduled_downgrade_at</code> column). BIL-03 just continues to close the current cycle; the downgrade fires when the framework's scanner picks up the scheduled action.	NEXT_CYCLE downgrade coordination
R-BIL-03-S-5	When SUB-WF-TERMINATE-01 terminates a Subscription mid-cycle, the cycle is closed as part of the termination saga (SUB-WF-TERMINATE-01 calls billing with <code>TERMINATION_INTENT</code> , and BIL-01 settles the final cycle as part of that). BIL-03 receives <code>SubscriptionTerminated</code> , marks the Subscription's <code>last_cycle_closed_at</code> to current time, and stops evaluating that Subscription.	SubscriptionTerminated event
R-BIL-03-S-6	When a Subscription is in any <code>PENDING_*</code> status (mid-workflow), BIL-03's scanner skips it and schedules re-evaluation for <code>now() + 1 hour</code> . Long-running <code>PENDING_*</code> (>4 hours) is flagged via <code>CycleSkippedDueToPendingTooLong</code> Kafka event for operator review — this indicates a stuck workflow.	Mid-workflow gating

Cycle alignment on workflow events

ID	Rule	Triggered on
R-BIL-03-V-1	BIL-03 consumes the following workflow events to keep its evaluation queue in sync: <code>SubscriptionActivated</code> , <code>SubscriptionPaused</code> , <code>SubscriptionResumed</code> , <code>SubscriptionUpgraded</code> , <code>SubscriptionDowngraded</code> , <code>SubscriptionRelocated</code> , <code>SubscriptionMigrated</code> , <code>SubscriptionTerminated</code> . Each event triggers a re-read of the Subscription's cycle anchor and a re-schedule of the next BIL-03 evaluation.	Workflow events
R-BIL-03-V-2	On <code>SubscriptionActivated</code> : schedule first cycle close at <code>current_cycle_end</code> (which is <code>current_cycle_start + cycle_period</code>). For pre-payment Subscriptions: also schedule the pre-notice and the pre-invoicing trigger to BIL-02 per R-BIL-03-P-2.	Activation event
R-BIL-03-V-3	BIL-03 maintains a single internal table <code>subscription_cycle_evaluation</code> keyed by <code>subscription_id</code> with columns: <code>next_evaluation_at</code> , <code>evaluation_type</code> (<code>STANDARD_CLOSE</code> / <code>PRE_NOTICE</code> / <code>PRE_INVOICING</code>), <code>last_evaluated_at</code> , <code>last_close_failure_code</code> , <code>failure_attempts</code> . This table drives the scanner queries and is updated on every scanner pass and on every workflow event consumption.	Scanner state

3. Module owner and impacted modules

Role	Module	What it does in this process
Owner	New billing-cycle-close module	Owns the cycle close scanner, the pre-notice scanner, the cycle evaluation table, the workflow event consumers, the cycle-close Kafka event publication, the internal REST endpoint for late-payment realignment.
Impacted (upstream events consumed)	Workflow Kafka events: SubscriptionActivated, SubscriptionPaused, SubscriptionResumed, SubscriptionUpgraded, SubscriptionDowngraded, SubscriptionRelocated, SubscriptionMigrated, SubscriptionTerminated	Source events keep the cycle evaluation table in sync with Subscription state.
Impacted (callees via REST)	SUB-LM-01 (subscription cycle attributes read + put -master- fields for cycle anchor advance), BIL-01 (payment status query for pre-payment cycles, late payment realignment endpoint), BIL-05 (wallet balance query and wallet debit for PREPAID cycle activation)	Receive synchronous REST calls from BIL-03's scanner and event handlers.
Impacted (downstream consumers via Kafka)	BIL-02 (consumes CycleClosed / CyclePartiallyClosed / CycleActivated for invoicing), BIL-04 (consumes CyclePaymentMissed for dunning), notification module (DD_NOT-01, planned, for pre-notice and customer-facing cycle events), CVM, analytics	Downstream modules driven by BIL-03's events.
Frontend	None directly. Operators view cycle close runs via admin console: dashboard with run history, per-Subscription drill-in showing cycle history, failed-cycle queue.	Read-only admin views.
Engines	No Drools. BIL-03 is a deterministic scanner — boundary detection is date math, action choice is a billing-mode switch.	—

4. Foundation references

DB → FOUNDATION_DB.md · Kafka → FOUNDATION_KAFKA.md · Auth → FOUNDATION_AUTH.md. Framework: SUB-WF-FRAMEWORK-01 (BIL-03 follows the scheduled scanner pattern documented there).

Technical design

Module placement

Layer	Where
Frontend	New admin Cycle Close console (operator view of cycle close runs, failed-cycle queue, per-Subscription cycle history).
Backend	New billing-cycle-close module on cluster C (revenue cluster, alongside billing, subscription, dunning).
Rules	None. Cycle close is deterministic Java/PHP logic.
Events consumed	8 workflow events (see Section 3).
Events emitted	6 events (see Events section).

Cycle close evaluation flow

[Embedded image omitted: Cycle close per-subscription flow]

Cycle anchor lifecycle per billing mode

[Embedded image omitted: Postpaid vs pre-payment cycle anchor]

Codebase

Java/Spring Boot layout

modules/billing-cycle-close/

```

src/main/java/com/sophix/billing/cyclecclose/
web/rest/
  CycleEvaluationResource.java      ← admin read + late-payment realign endpoints
  CycleCloseRunResource.java       ← run history
  dto/...
service/
  CycleCloseService.java           ← core per-Subscription cycle close logic
  CycleBoundaryDetector.java       ← date math for boundary detection
  PrePaymentCycleEvaluator.java    ← pre-payment-specific evaluation (payment status check, cycle activation)
  PrepaidWalletCycleEvaluator.java ← PREPAID-specific evaluation (wallet balance check, wallet debit, cycle activation)
  CycleAnchorAdvancer.java         ← updates subscription cycle anchor via SUB-LM-01
  LateCyclePaymentHandler.java     ← handles late-payment cycle realignment (R-BIL-03-P-4)
domain/
  CycleEvaluation.java             ← entity (one row per active Subscription)
  CycleEvaluationType.java         ← enum STANDARD_CLOSE | PRE_NOTICE | PRE_INVOICING
  CycleCloseRun.java              ← entity
  CycleCloseOutcome.java          ← enum CLOSED | ACTIVATED | PAYMENT_MISSED | SKIPPED_SUSPENDED | SKIPPED
  CycleCloseFailureReason.java
repository/
  CycleEvaluationRepository.java
  CycleCloseRunRepository.java
scanner/
  CycleCloseScanner.java           ← @Scheduled(fixedDelay = 1800000) every 30 min
  PrePaymentPreNoticeScanner.java ← @Scheduled(cron = "0 3 * * ?") daily 03:00
kafka/
  SubscriptionActivatedConsumer.java
  SubscriptionPausedConsumer.java
  SubscriptionResumedConsumer.java
  SubscriptionUpgradedConsumer.java
  SubscriptionDowngradedConsumer.java
  SubscriptionRelocatedConsumer.java
  SubscriptionMigratedConsumer.java
  SubscriptionTerminatedConsumer.java
event/
  CycleClosedEventPublisher.java
  CyclePartiallyClosedEventPublisher.java
  CycleActivatedEventPublisher.java
  CyclePaymentMissedEventPublisher.java
  CyclePaymentDueSoonEventPublisher.java
  WalletLowBeforeCycleEventPublisher.java
  CycleCloseFailedEventPublisher.java
  CycleSkippedDueToPendingTooLongEventPublisher.java
api/
  SubscriptionStateApi.java        ← REST client to SUB-LM-01
  BillingPaymentApi.java           ← REST client to BIL-01 for payment status (PREPAYMENT)
  WalletApi.java                  ← REST client to BIL-05 for wallet balance query + debit (PREPAID)
src/main/resources/
  config/liquibase/changelog/billing-cycle-close/
    001_subscription_cycle_evaluation.xml
    002_cycle_close_run.xml

```

Laravel/PHP parallel layout

```

sophix/billing-cycle-close/
src/Http/Controllers/
  CycleEvaluationController.php
  CycleCloseRunController.php
src/Http/Requests/
  LatePaymentRealignRequest.php
src/Service/
  CycleCloseService.php
  CycleBoundaryDetector.php
  PrePaymentCycleEvaluator.php
  PrepaidWalletCycleEvaluator.php ← PREPAID-specific evaluation
  CycleAnchorAdvancer.php
  LateCyclePaymentHandler.php
src/Domain/
  CycleEvaluation.php
  CycleEvaluationType.php
  CycleCloseRun.php
  CycleCloseOutcome.php
  CycleCloseFailureReason.php
src/Repository/
  CycleEvaluationRepository.php
  CycleCloseRunRepository.php
src/Console/
  CycleCloseCommand.php           ← scheduled via app/Console/Kernel.php everyThirtyMinutes()
  PrePaymentPreNoticeCommand.php ← scheduled dailyAt('03:00')
src/Kafka/
  SubscriptionActivatedListener.php
  SubscriptionPausedListener.php
  SubscriptionResumedListener.php
  SubscriptionUpgradedListener.php

```

```

SubscriptionDowngradedListener.php
SubscriptionRelocatedListener.php
SubscriptionMigratedListener.php
SubscriptionTerminatedListener.php
src/Event/
  CycleClosedEventPublisher.php
  CyclePartiallyClosedEventPublisher.php
  CycleActivatedEventPublisher.php
  CyclePaymentMissedEventPublisher.php
  CyclePaymentDueSoonEventPublisher.php
  WalletLowBeforeCycleEventPublisher.php
  CycleCloseFailedEventPublisher.php
  CycleSkippedDueToPendingTooLongEventPublisher.php
src/Api/
  SubscriptionStateApi.php
  BillingPaymentApi.php
  WalletApi.php
src/Providers/
  BillingCycleCloseServiceProvider.php
routes/
  api.php
database/migrations/
  2026_08_01_create_subscription_cycle_evaluation_table.php
  2026_08_01_create_cycle_close_run_table.php

```

← BIL-05 wallet balance + debit (PREPAID)

Foundation-specific configurations

Foundation	Configuration
DB	New tables on cluster C (revenue): subscription_cycle_evaluation (one row per active Subscription, ~200k rows for Wananchi group target), cycle_close_run (one row per scanner run, monthly partitioned per FOUNDATION_DB.md). _audit shadow tables on both.
Drools	None.
Kafka	Consumes 8 workflow events, emits 8 cycle-related events (added WalletLowBeforeCycle for PREPAID pre-notice) (all on cluster C Kafka brokers per FOUNDATION_KAFKA.md).
Auth	Roles: BILLING_INTERNAL (BIL-01's late-payment realign call), CYCLE_ADMIN (admin read + manual intervention), CYCLE_OPERATOR (read-only).

Frontend

Screen	Purpose
Admin Cycle Close Console — Run History	List of recent cycle_close_run records with subscriptions_evaluated / closed / failed counts
Admin Cycle Close Console — Failed Cycle Queue	Subscriptions with failure_attempts >= 3, with retry/skip/escalate actions
Admin Cycle Close Console — Subscription Drill-in	One Subscription's cycle history: past close events, current cycle window, next evaluation timestamp, any pre-notice / pre-invoicing scheduled
Admin Cycle Close Console — Manual Trigger (role CYCLE_ADMIN)	Force-close a specific Subscription's cycle for exceptional cases (audited)

Data model

subscription_cycle_evaluation

One row per active Subscription. Drives the scanner queue.

Column	Type	Notes
subscription_id	TEXT PK	One row per Subscription
billing_mode	TEXT NOT NULL	Cached from SUB-LM-01 for scanner-side filtering (POSTPAID / PREPAYMENT / PREPAID)
operator_code	TEXT NOT NULL	Cached for tenant-scoped queries
next_evaluation_at	TIMESTAMPZ NOT NULL	When BIL-03 should next look at this Subscription
next_evaluation_type	TEXT NOT NULL	STANDARD_CLOSE / PRE_NOTICE / PRE_INVOICING
current_cycle_end	TIMESTAMPZ NOT NULL	Cached from SUB-LM-01; refreshed on each event consumption + each scanner run
last_evaluated_at	TIMESTAMPZ NULL	When last evaluated

Column	Type	Notes
last_cycle_closed_window_end	TIMESTAMPTZ NULL	Window end of the last successfully closed cycle (for idempotency per R-BIL-03-C-5)
last_pre_notice_fired_at	TIMESTAMPTZ NULL	Most recent pre-notice firing (for dedup on daily scanner)
last_close_failure_code	TEXT NULL	Set on failure
last_close_failure_at	TIMESTAMPTZ NULL	
failure_attempts	INTEGER NOT NULL DEFAULT 0	Resets on success
created_at	TIMESTAMPTZ NOT NULL	When this row was created (typically on SubscriptionActivated)
archived_at	TIMESTAMPTZ NULL	Set when Subscription terminates; row is logically removed from scanner queue

Indexes:

- (next_evaluation_at, next_evaluation_type) — scanner query
- (billing_mode, next_evaluation_at) — billing-mode-scoped scanner queries
- (subscription_id) PK
- partial index WHERE archived_at IS NULL to keep scanner queries fast

_audit shadow table per FOUNDATION_DB.md.

cycle_close_run

One row per scanner invocation. Audit + observability.

Column	Type	Notes
run_id	TEXT PK	ULID
scanner_type	TEXT NOT NULL	STANDARD_CLOSE / PRE_NOTICE
node_id	TEXT NOT NULL	Which application node ran this
started_at	TIMESTAMPTZ NOT NULL	
completed_at	TIMESTAMPTZ NULL	
subscriptions_evaluated	INTEGER NOT NULL DEFAULT 0	
subscriptions_closed	INTEGER NOT NULL DEFAULT 0	Successful POSTPAID closes
subscriptions_activated	INTEGER NOT NULL DEFAULT 0	Successful PREPAYMENT cycle activations
subscriptions_payment_missed	INTEGER NOT NULL DEFAULT 0	PREPAYMENT cycles fired CyclePaymentMissed
subscriptions_skipped	INTEGER NOT NULL DEFAULT 0	Skipped (SUSPENDED, PENDING_*, etc.)
subscriptions_failed	INTEGER NOT NULL DEFAULT 0	Internal failure
pre_notices_fired	INTEGER NOT NULL DEFAULT 0	Pre-payment pre-notice scanner only
error_summary	JSONB NULL	Aggregated error codes + counts

Monthly partitioned per FOUNDATION_DB.md. P10Y retention for compliance / capacity planning.

Cycle close service — core logic

```

@Service
public class CycleCloseService {

    @Transactional
    public CycleCloseOutcome closeOne(String subscriptionId, CycleCloseRun run) {
        CycleEvaluation eval = evaluationRepo.lockBySubscriptionId(subscriptionId);
        SubscriptionSnapshot snapshot = subscriptionStateApi.fetchSnapshot(subscriptionId);

        // Gate per R-BIL-03-C-4 and S-6
        if (!isCloseEligible(snapshot)) {
            return handleSkip(eval, snapshot);
        }

        // Idempotency per R-BIL-03-C-5
        if (eval.getLastCycleClosedWindowEnd() != null
            && !snapshot.getCycleWindow().getEnd().isAfter(eval.getLastCycleClosedWindowEnd())) {
            return CycleCloseOutcome.SKIPPED_IDEMPOTENT;
        }
    }
}

```

```

    }

    try {
        switch (snapshot.getBillingMode()) {
            case POSTPAID:
                return closePostpaid(subscriptionId, snapshot, eval);
            case PREPAID:
                return evaluatePrepaidWallet(subscriptionId, snapshot, eval);
            case PREPAYMENT:
                return evaluatePrePayment(subscriptionId, snapshot, eval);
            default:
                throw new IllegalStateException("Unknown billing mode");
        }
    } catch (Exception e) {
        return handleFailure(eval, e);
    }
}

private CycleCloseOutcome closePostpaid(String subscriptionId, SubscriptionSnapshot snapshot,
                                         CycleEvaluation eval) {
    cycleClosedEventPublisher.publish(buildCycleClosedEvent(snapshot));
    cycleAnchorAdvancer.advance(subscriptionId, snapshot);
    eval.recordClosed(snapshot.getCycleWindow().getEnd());
    evaluationRepo.save(eval);
    return CycleCloseOutcome.CLOSED;
}

private CycleCloseOutcome evaluatePrepaidWallet(String subscriptionId, SubscriptionSnapshot snapshot,
                                                CycleEvaluation eval) {
    BigDecimal cycleCharge = computeCycleCharge(snapshot);
    WalletBalance balance = walletApi.fetchBalance(subscriptionId);

    if (balance.getAmount().compareTo(cycleCharge) >= 0) {
        // Wallet sufficient – debit and activate
        String idempotencyKey = "cycle-debit-" + subscriptionId + "-" + snapshot.getCycleWindow().getEnd().toString();
        WalletDebitResult debitResult = walletApi.debit(subscriptionId, cycleCharge, idempotencyKey);
        cycleActivatedEventPublisher.publish(buildCycleActivatedPrepaidEvent(snapshot, debitResult));
        cycleAnchorAdvancer.advance(subscriptionId, snapshot);
        eval.recordClosed(snapshot.getCycleWindow().getEnd());
        evaluationRepo.save(eval);
        return CycleCloseOutcome.ACTIVATED;
    }

    // Wallet insufficient → CyclePaymentMissed (PREPAID variant)
    cyclePaymentMissedEventPublisher.publish(buildCyclePaymentMissedPrepaidEvent(snapshot, balance, cycleCharge));
    eval.recordPaymentMissed(snapshot.getCycleWindow().getEnd());
    evaluationRepo.save(eval);
    // Note: cycle anchor NOT advanced (R-BIL-03-W-3) – BIL-04 advances on wallet topup recovery (R-BIL-04-R-8)
    return CycleCloseOutcome.PAYMENT_MISSED;
}

private CycleCloseOutcome evaluatePrePayment(String subscriptionId, SubscriptionSnapshot snapshot,
                                              CycleEvaluation eval) {
    String invoiceId = snapshot.getNextCycleChargeInvoiceId();
    PaymentStatus paymentStatus = billingPaymentApi.fetchInvoicePaymentStatus(invoiceId);

    if (paymentStatus == PaymentStatus.PAID) {
        cycleActivatedEventPublisher.publish(buildCycleActivatedPrepaymentEvent(snapshot));
        cycleAnchorAdvancer.advance(subscriptionId, snapshot);
        eval.recordClosed(snapshot.getCycleWindow().getEnd());
        evaluationRepo.save(eval);
        return CycleCloseOutcome.ACTIVATED;
    }

    // UNPAID or PARTIALLY_PAID → CyclePaymentMissed (PREPAYMENT variant)
    cyclePaymentMissedEventPublisher.publish(buildCyclePaymentMissedPrepaymentEvent(snapshot, paymentStatus));
    eval.recordPaymentMissed(snapshot.getCycleWindow().getEnd());
    evaluationRepo.save(eval);
    // Note: cycle anchor NOT advanced (R-BIL-03-P-3) – BIL-01 advances on PaymentApplied (PREPAYMENT recovery)
    return CycleCloseOutcome.PAYMENT_MISSED;
}
}

```

The Laravel/PHP equivalent follows the same logic with `DB::transaction()` closure and Throwable handling, per the dual-stack pattern in SUB-WF-FRAMEWORK-01.

Events (Kafka)

sophix.billing.cycle.closed (postpaid)

Emitted at every postpaid cycle close. BIL-02 consumes to generate the invoice.

```
{
  "eventType": "CycleClosed",
  "aggregateId": "sub_01HX...",
  "timestamp": "2026-08-31T23:59:59Z",
  "payload": {
    "subscriptionId": "sub_01HX...",
    "customerId": "cust_01HX...",
    "operatorCode": "WANANCHI_KE",
    "billingMode": "POSTPAID",
    "currency": "KES",
    "cycleStart": "2026-08-01T00:00:00Z",
    "cycleEnd": "2026-08-31T23:59:59Z",
    "cycleTotalDays": 31,
    "cycleAnchorDay": 1,
    "cycleFrequencyMonths": 1,
    "packageRef": "pkg_internet_100mbps",
    "packageVersion": 3,
    "homepassId": "hp_01HX...",
    "homepassFranchiseId": "fr_nairobi_north",
    "activeRestrictionCodes": [],
    "closedAt": "2026-08-31T23:59:59Z"
  }
}
```

sophix.billing.cycle.activated (PREPAID wallet-debit success OR PREPAYMENT cycle-invoice-paid)

Emitted when a PREPAID or PREPAYMENT cycle becomes active. For PREPAID: after BIL-03 successfully debits the wallet via BIL-05. For PREPAYMENT: after BIL-03 confirms the cycle invoice is paid via BIL-01. BIL-02 consumes for record-keeping; notification consumes for "cycle started" confirmation.

Example for a Wananchi KE PREPAID wallet cycle:

```
{
  "eventType": "CycleActivated",
  "aggregateId": "sub_01HZ...",
  "timestamp": "2026-09-01T00:00:00Z",
  "payload": {
    "subscriptionId": "sub_01HZ...",
    "customerId": "cust_01HZ...",
    "operatorCode": "WANANCHI_KE",
    "billingMode": "PREPAID",
    "currency": "KES",
    "cycleStart": "2026-09-01T00:00:00Z",
    "cycleEnd": "2026-09-30T23:59:59Z",
    "cycleChargeAmount": 4500.00,
    "walletDebitedAmount": 4500.00,
    "walletBalanceAfter": 1500.00,
    "recoveryFromDunning": false,
    "lateActivation": false,
    "originalCycleStart": null,
    "activatedAt": "2026-09-01T00:00:00Z"
  }
}
```

Example for a Yas SN PREPAYMENT cycle:

```
{
  "eventType": "CycleActivated",
  "aggregateId": "sub_01HY...",
  "timestamp": "2026-08-01T00:00:00Z",
  "payload": {
    "subscriptionId": "sub_01HY...",
    "customerId": "cust_01HY...",
    "operatorCode": "YAS_SN",
    "billingMode": "PREPAYMENT",
    "currency": "XOF",
    "cycleStart": "2026-08-01T00:00:00Z",
    "cycleEnd": "2026-08-31T23:59:59Z",
    "cycleChargeInvoiceId": "inv_01HY...",
    "cycleChargeAmount": 25000.00,
    "lateActivation": false,
    "originalCycleStart": null,
    "activatedAt": "2026-08-01T00:00:00Z"
  }
}
```

For late activation (R-BIL-03-P-4) or PREPAID dunning recovery (R-BIL-03-W-4), lateActivation: true or recoveryFromDunning: true and originalCycleStart populated.

sophix.billing.cycle.paymentmissed (PREPAID wallet-insufficient OR PREPAYMENT cycle-invoice-unpaid)

Emitted at cycle boundary when either: a PREPAID Subscription's wallet has insufficient balance for the cycle charge (per R-BIL-03-W-1), OR a PREPAYMENT Subscription's upcoming cycle invoice is unpaid (per R-BIL-03-P-1). Same event type used in both cases — the `billingMode` field on the payload distinguishes them. BIL-04 consumes to enter dunning.

Example for a Wananchi KE PREPAID wallet-insufficient:

```
{
  "eventType": "CyclePaymentMissed",
  "aggregateId": "sub_01HZ...",
  "timestamp": "2026-09-01T00:00:00Z",
  "payload": {
    "subscriptionId": "sub_01HZ...",
    "customerId": "cust_01HZ...",
    "operatorCode": "WANANCHI_KE",
    "billingMode": "PREPAID",
    "currency": "KES",
    "missedCycleStart": "2026-09-01T00:00:00Z",
    "missedCycleEnd": "2026-09-30T23:59:59Z",
    "cycleChargeAmount": 4500.00,
    "walletBalanceAtBoundary": 1200.00,
    "shortfallAmount": 3300.00,
    "cycleChargeInvoiceId": null,
    "paymentStatus": "WALLET_INSUFFICIENT",
    "detectedAt": "2026-09-01T00:00:00Z"
  }
}
```

Example for a Yas SN PREPAYMENT cycle-invoice-unpaid:

```
{
  "eventType": "CyclePaymentMissed",
  "aggregateId": "sub_01HY...",
  "timestamp": "2026-08-01T00:00:00Z",
  "payload": {
    "subscriptionId": "sub_01HY...",
    "customerId": "cust_01HY...",
    "operatorCode": "YAS_SN",
    "billingMode": "PREPAYMENT",
    "currency": "XOF",
    "missedCycleStart": "2026-08-01T00:00:00Z",
    "missedCycleEnd": "2026-08-31T23:59:59Z",
    "cycleChargeInvoiceId": "inv_01HY...",
    "expectedAmount": 25000.00,
    "paidAmount": 0.00,
    "paymentStatus": "UNPAID",
    "detectedAt": "2026-08-01T00:00:00Z"
  }
}
```

Note `paymentStatus` may be UNPAID / PARTIALLY_PAID (PREPAYMENT) or WALLET_INSUFFICIENT (PREPAID). BIL-04 enters dunning in all cases per R-BIL-03-P-1 and R-BIL-03-W-1.

`sophix.billing.cycle.paymentduesoon` (PREPAYMENT pre-notice)

Daily scanner output for PREPAYMENT Subscriptions. Consumed by notification module for "your cycle starts in N days — pay now" comms.

```
{
  "eventType": "CyclePaymentDueSoon",
  "aggregateId": "sub_01HY...",
  "timestamp": "2026-07-29T03:00:00Z",
  "payload": {
    "subscriptionId": "sub_01HY...",
    "customerId": "cust_01HY...",
    "operatorCode": "YAS_SN",
    "billingMode": "PREPAYMENT",
    "currency": "XOF",
    "upcomingCycleStart": "2026-08-01T00:00:00Z",
    "upcomingCycleChargeInvoiceId": "inv_01HY...",
    "upcomingCycleAmount": 25000.00,
    "daysUntilCycleStart": 3,
    "currentPaymentStatus": "UNPAID"
  }
}
```

`sophix.billing.cycle.walletlowbeforecycle` (PREPAID pre-notice)

Daily scanner output for PREPAID Subscriptions with insufficient wallet balance for the upcoming cycle (per R-BIL-03-W-5). Consumed by notification module for "your wallet is low — top up X KES before [date] to avoid service interruption" comms.

```
{
  "eventType": "WalletLowBeforeCycle",
  "aggregateId": "sub_01HZ...",
  "timestamp": "2026-08-29T03:00:00Z",
  "payload": {
    "subscriptionId": "sub_01HZ...",
    "customerId": "cust_01HZ...",
    "operatorCode": "WANANCHI_KE",
    "billingMode": "PREPAID",
    "currency": "KES",
    "upcomingCycleStart": "2026-09-01T00:00:00Z",
    "upcomingCycleChargeAmount": 4500.00,
    "currentWalletBalance": 1200.00,
    "shortfallAmount": 3300.00,
    "daysUntilCycleStart": 3
  }
}
```

sophix.billing.cycle.partiallyclosed (pause prorate)

Emitted when SUB-WF-PAUSE-01 pauses a Subscription mid-cycle with `pauses_bill_cycle = true`. BIL-02 consumes to issue the prorated invoice.

```
{
  "eventType": "CyclePartiallyClosed",
  "aggregateId": "sub_01HX...",
  "timestamp": "2026-08-15T14:00:00Z",
  "payload": {
    "subscriptionId": "sub_01HX...",
    "customerId": "cust_01HX...",
    "operatorCode": "WANANCHI_KE",
    "billingMode": "POSTPAID",
    "currency": "KES",
    "cycleStart": "2026-08-01T00:00:00Z",
    "partialCycleEnd": "2026-08-15T14:00:00Z",
    "fullCycleEnd": "2026-08-31T23:59:59Z",
    "daysServed": 15,
    "cycleTotalDays": 31,
    "proratePercentage": 0.4839,
    "pauseStartedAt": "2026-08-15T14:00:00Z",
    "pauseReasonCode": "CUSTOMER_REQUEST"
  }
}
```

sophix.billing.cycle.closefailed

Emitted when a Subscription fails close 3 times in a row. Surfaces for operator review.

```
{
  "eventType": "CycleCloseFailed",
  "aggregateId": "sub_01HX...",
  "timestamp": "2026-09-01T01:30:00Z",
  "payload": {
    "subscriptionId": "sub_01HX...",
    "cycleEnd": "2026-08-31T23:59:59Z",
    "failureCode": "SUB_LM_TIMEOUT",
    "failureDetail": "SUB-LM-01 fetchSnapshot timed out after 5s across 3 attempts",
    "attempts": 3,
    "requiresOperatorAttention": true
  }
}
```

sophix.billing.cycle.skippedduetopendingtoolong

Emitted when a Subscription has been in `PENDING_*` for > 4 hours and is therefore stuck. Surfaces for operator review.

```
{
  "eventType": "CycleSkippedDueToPendingTooLong",
  "aggregateId": "sub_01HX...",
  "timestamp": "2026-09-01T06:00:00Z",
  "payload": {
    "subscriptionId": "sub_01HX...",
    "pendingStatus": "PENDING_UPGRADE",
    "pendingSince": "2026-09-01T01:30:00Z",
    "hoursPending": 4.5,
    "cycleEndWasDue": "2026-09-01T00:00:00Z"
  }
}
```

APIs (in this module)

Read endpoints


```

GET /api/cycle-evaluations/{subscriptionId}          ← current cycle evaluation row
GET /api/cycle-evaluations?evaluation_type={type}&next_before={ts}&limit=50 ← admin scanner-queue view
GET /api/cycle-close-runs?limit=50                  ← recent runs
GET /api/cycle-close-runs/{run_id}                  ← run drill-in
GET /api/cycle-evaluations/failed                    ← failed-cycle queue

```

Admin actions (role CYCLE_ADMIN)

```

POST /api/cycle-evaluations/{subscriptionId}/force-close ← manually trigger cycle close (audited)
POST /api/cycle-evaluations/{subscriptionId}/skip-current ← skip the current cycle's close (rare, audited)
POST /api/cycle-evaluations/{subscriptionId}/reset-failures ← clear failure counter after manual fix

```

Internal endpoints (called by BIL-01 and BIL-04)

```

POST /api/cycles/{subscriptionId}/realign-on-late-payment ← BIL-01 calls when PREPAYMENT late-payment arrives (R-BIL-03)
POST /api/cycles/{subscriptionId}/advance-cycle-on-recovery ← BIL-04 calls during PREPAID wallet-topup auto-recovery (R-BIL-04)

```

Both endpoints accept the same request body shape:

```

{
  "eventTimestamp": "2026-09-05T11:00:00Z",
  "triggeringEventRef": "inv_01HY... | wallet_topup_01HZ...",
  "idempotencyKey": "late-realign-sub_01HY-2026-08 | advance-recovery-sub_01HZ-2026-09"
}

```

Response 200 — cycle anchor advanced, CycleActivated event emitted with `lateActivation: true` (PREPAYMENT) or `recoveryFromDunning: true` (PREPAID), `originalCycleStart` populated for audit.

Both endpoints are role-gated to BILLING_INTERNAL and idempotent on the supplied `idempotencyKey`.

Cross-DD coordination

Touched	What BIL-03 owns	What the other DD owns
SUB-WF-FRAMEWORK-01	Follows scheduled scanner pattern.	Owns the framework.
SUB-LM-01	Reads cycle attributes via <code>fetchSnapshot</code> and <code>computeCycleWindow</code> . Writes cycle anchor advance via <code>put-master-fields</code> . Maintains its own per-Subscription <code>cycle_evaluation</code> row (cached values for scanner-side filtering).	Owns Subscription master fields including all cycle attributes.
SUB-WF-* workflow events	Consumes 8 workflow events to keep <code>cycle_evaluation</code> in sync.	Each workflow DD emits its event with full cycle context (per R-SUB-WF-FRAMEWORK-01-NF-2).
BIL-02 (Invoicing)	Emits <code>CycleClosed</code> (POSTPAID invoice trigger), <code>CycleActivated</code> (PREPAID + PREPAYMENT record-keeping), <code>CyclePartiallyClosed</code> (prorated invoice trigger at pause). BIL-02 writes <code>next_cycle_charge_invoice_id</code> to SUB-LM-01 for PREPAYMENT cycles (per R-BIL-03-P-2).	Owns invoice generation, due date evaluation, <code>InvoiceOverdue</code> emission.
BIL-01 (Charging Engine)	Queries payment status via REST for PREPAYMENT cycle evaluation. Accepts late-payment realign calls from BIL-01 (per R-BIL-03-P-4).	Owns charge accounting, payment recording, cycle realignment on PREPAYMENT late payment.
BIL-04 (Dunning)	Emits <code>CyclePaymentMissed</code> consumed by BIL-04 (covers both PREPAID wallet-insufficient and PREPAYMENT cycle-unpaid). Accepts <code>advance-cycle-on-recovery</code> calls from BIL-04 during PREPAID wallet-topup auto-recovery (per R-BIL-03-W-4 and R-BIL-04-R-8).	Owns dunning state machine.
BIL-05 (Wallet & Topup)	For PREPAID Subscriptions: queries wallet balance via REST at cycle boundary (R-BIL-03-W-1); calls wallet debit at cycle activation. Queries balance again in the daily pre-notice scanner (R-BIL-03-W-5).	Owns the wallet, topup channels, balance tracking, debit/credit operations.
notification module (DD_NOT-01, planned)	Emits 8 cycle-related events (added <code>WalletLowBeforeCycle</code> for PREPAID pre-notice) with all fields a notification template needs.	Consumes events. Expected customer comms: PREPAYMENT cycle pre-notice (3 days before cycle, "pay invoice"), PREPAID wallet-low pre-notice (3 days before cycle, "top up wallet"), cycle activated confirmation, cycle payment missed alert. Cycle invoice notification (for POSTPAID and PREPAYMENT) flows via BIL-02 not BIL-03.

Dependencies

Dependency	Owner	Status
SUB-WF-FRAMEWORK-01	subscription team	✓ DD complete (v0.9)
SUB-LM-01 (cycle attributes, put-master-fields, fetchSnapshot)	subscription team	✓ DD complete (v0.8)
All SUB-WF-* DDs (event publishers BIL-03 consumes)	subscription team	✓ DDs complete (v0.9)
BIL-01 (payment status query API, late-payment realign caller)	billing team	■ DD partial (v0.8 — payment status API and late-realign caller not yet specified)
BIL-02 (consumes CycleClosed/CyclePartiallyClosed/CycleActivated, writes next_cycle_charge_invoice_id)	billing team	■ pending DD
BIL-04 (consumes CyclePaymentMissed for both PREPAID and PREPAYMENT; calls advance-cycle-on-recovery for PREPAID auto-recovery)	billing team	✓ DD complete (v1.4)
BIL-05 (Wallet & Topup — wallet balance query and debit API for PREPAID cycle activation)	billing team	■ pending DD (Wave 1 backlog)
FOUNDATION_DB.md	infra team	✓ Documented
FOUNDATION_KAFKA.md	infra team	✓ Documented
FOUNDATION_AUTH.md (CYCLE_ADMIN, CYCLE_OPERATOR, BILLING_INTERNAL roles)	infra team	✓ Documented
DD_NOT-01 Notification Service	notification team	■ pending DD (Wave 1 backlog)

B — Current TZ

To be filled in after codebase review. No claims about what the TZ-deployed Sophix codebase does for cycle close are recorded here until the codebase has been read directly.

The cartography phase (per the pending plan) will produce this content. Once the codebase is available, this section will be filled with verified statements about which cycle-close-related capabilities exist and which do not — replacing this placeholder.

C — V3 design rationale

This DD's V3 design choices follow from the workshop requirements (section A) and the cross-operator billing-model variation (POSTPAID Wananchi, PREPAID wallet Wananchi, PREPAYMENT Yas SN). The design intent:

What V3 introduces

1. **Dedicated billing-cycle-close module** separate from invoice generation. Cycle-close emits events; invoice generation consumes them. Decoupling allows other consumers (BIL-04 dunning, BIL-05 wallet, notifications) to react to the same cycle events without reaching into invoice-generation internals.
2. **Three billing models on day 1**: POSTPAID (Wananchi some customers), PREPAID wallet (majority of Wananchi customers), PREPAYMENT (Yas SN). Same module, three behaviors driven by the Subscription's `billing_mode` field.
3. **Eight distinct cycle events** for downstream consumption — including `WalletLowBeforeCycle` for PREPAID pre-notice and `CycleActivated` (multi-mode) for both PREPAID wallet-debit success and PREPAYMENT cycle-paid.
4. **PREPAID wallet integration**: BIL-03 queries BIL-05 for balance at cycle boundary, calls BIL-05 to debit wallet on successful activation, fires `CyclePaymentMissed` on insufficient balance.
5. **PREPAID auto-recovery coordination**: BIL-04 calls BIL-03's `advance-cycle-on-recovery` endpoint after a wallet topup brings the customer back to good standing.
6. **Workflow event consumption** for cycle evaluation sync — 8 workflow event consumers keep cycle evaluations aligned with subscription lifecycle changes.
7. **Mid-cycle prorate via CyclePartiallyClosed** driving immediate prorated invoicing at pause.
8. **Pre-notice scanners** for both PREPAYMENT (cycle invoice due soon) and PREPAID (wallet low before cycle).
9. **Late-payment realignment** for PREPAYMENT (realign-on-late-payment) and recovery-cycle-advance for PREPAID (advance-cycle-on-recovery) — two distinct internal endpoints for the two billing models.

10. **Idempotency** on (subscription_id, cycle_window_end) per R-BIL-03-C-5; wallet debit idempotency per R-BIL-03-W-2.
11. **Per-Subscription** cycle_evaluation **table** with retry-on-failure and operator-review escalation.
12. cycle_close_run **audit table** with P10Y retention.
13. ****Stuck PENDING_*** workflow detection** via CycleSkippedDueToPendingTooLong event.
14. **Per-operator + per-billing-mode behavior** configured via the Subscription's billing_mode field; same module code serves all four operating countries.

Migration approach (TZ-baseline-agnostic)

1. Deploy billing-cycle-close module with two tables on cluster C. Requires BIL-05 (Wallet & Topup) deployed first since BIL-03 calls BIL-05 for PREPAID cycle activation — BIL-05 is a hard dependency.
2. **Backfill phase** (one-time at V3 cutover): for each currently-active Subscription, create a subscription_cycle_evaluation row with next_evaluation_at = current_cycle_end and next_evaluation_type = STANDARD_CLOSE. For Wananchi PREPAID Subscriptions: also create the daily wallet-low pre-notice entry. For Yas SN PREPAYMENT Subscriptions: also schedule the PRE_INVOICING and PRE_NOTICE entries. Cycle attributes come from SUB-LM-01 backfill (done as part of SUB-LM-01 migration). Billing mode comes from the V3 Subscription master record.
3. Deploy Kafka consumers for SUB-WF-* events. These start populating the cycle evaluation table for new Subscriptions.
4. **Shadow mode:** BIL-03 runs alongside whatever existing cycle-close mechanism the redeployed codebase has. BIL-03 scans, emits events, but BIL-02 (still in pre-cutover mode) ignores them. Operators verify cycle detection accuracy on the new path — including the PREPAID wallet-debit dry-run mode (BIL-05 supports a dry-run debit that does not actually deduct balance, for shadow-mode verification). Concrete details of the existing mechanism will be confirmed in the cartography phase.
5. Cutover: BIL-02 (new V3 mode) switches to consuming BIL-03's events. BIL-04 starts consuming CyclePaymentMissed and WalletLowBeforeCycle. Pre-cutover mechanism is disabled. BIL-05 wallet debits go live.
6. Per-operator activation order (recommended): start with Wananchi POSTPAID (smallest segment, simplest), then Wananchi PREPAID (largest segment, most complex), then Yas SN PREPAYMENT (separate operator, isolatable).

Mapping to TZ baseline (section B) — what V3 adds vs what the redeployed codebase already supports — will be filled in after codebase review.

14. Operator admin console for cycle close investigation and manual intervention.

V3.2 backlog (deferred)

Feature	When it would matter	V3.2 path
Mid-cycle billing mode conversion (POSTPAID ↔ PREPAID)	If customer credit profile improves/degrades during life of Subscription	Add workflow to terminate-and-recreate or build dedicated mode-conversion workflow
Customer-level cycle aggregation	If a customer's multiple Subscriptions need to share a cycle anchor (consolidated bill)	Add customer_cycle_evaluation table
Configurable per-Subscription pre-notice timing	If marketing wants A/B testing on pre-notice lead time	Move pre_notice_window_days from operator default to per-Subscription override
Predictive payment likelihood (pre-payment / wallet-low)	If ML can predict which customers are likely to miss payment or run out of wallet	Integrate with CVM/analytics ML output for proactive outreach
Cycle close burst handling	If 200k+ Subscriptions all roll on the same cycle anchor day and scanner backlog grows	Pre-emptive scheduling (jitter the next_evaluation_at within ±2 hours to spread load)
Wallet auto-debit on partial topup	If customer tops up multiple small amounts that together cover the cycle	Extend R-BIL-03-W-3 to allow partial accumulation across topup events (currently single-topup-must-cover)